Traffic Forecasting Through A GAT-GRU Hybrid Architecture

Justin Lam
Department of Computer Science
Yale University
New Haven, CT 06511
justin.lam@yale.edu

Abstract

Traffic forecasting is important to many facets of urban planning, particularly real-time traffic management and routing. This report proposes a novel GAT-GRU architecture to perform traffic forecasting. The model combines a Graph Attention Network (GAT) with a Gated Recurrent Unit (GRU) network to learn both spatial and temporal dependencies. Experiments show that the model can outperform multiple state-of-the-art baselines, highlighting the potential of hybrid attention-based graph convolution and recurrent architectures for traffic forecasting.

1 Introduction

As urban planning grows more complex and automated driving becomes more prevalent, the value of traffic forecasting has become more apparent. Traffic forecasting, in essence, is using current traffic conditions to predict future traffic conditions for a given road network. In particular, short-term traffic forecasting can be important for intelligent traffic control and road navigation.

Traffic forecasting is especially suited to prediction using a hybrid graph neural network (GNN) and recurrent neural network (RNN) approach, because there are both spatial and temporal dependencies—traffic at a given location is related to both surrounding traffic and previous traffic.

My proposed architecture is the GAT-GRU, which is a combination of a Graph Attention Network (GAT) [10] and a Gated Recurrent Unit (GRU) network [4]. This enables the model to learn both spatial and temporal dependencies of traffic graphs. In particular, the attention mechanism in the GAT enables the model to learn which neighboring traffic conditions have the greatest effect on the traffic at a given location, which allows it to capture the spatial dependencies of traffic more effectively.

My model was able to outperform or perform comparably to existing models on the PEMS04 and PEMS08 datasets [5]. The experiment results show that the GAT-GRU model is capable of effective traffic forecasting. It demonstrates the effectiveness of graph attention mechanisms for this problem.

I also observed that the model was able to generalize much better on short-term predictions (15 minutes as opposed to 60 minutes), suggesting that it is more suited for short-term or even real-time traffic forecasting rather than long-term.

2 Related Works

2.1 TGCN

The Temporal Graph Convolutional Network [14] is a relatively lightweight architecture that has been used to model traffic data. It feeds output from a Graph Convolutional Network (GCN) into a GRU to learn both spatial and temporal dependencies. Its advantage is its relative simplicity, which

allows for relatively fast implementation, training, and computation. Its disadvantage, however, is that it isn't as capable of capturing more complex spatial dependencies. In particular, compared to my proposed GAT-GRU architecture, it lacks the attention mechanism, which can lead to worse performance because it treats all neighbors equally.

2.2 DCRNN

The Diffusion Convolutional Recurrent Neural Network [7] doesn't use a GCN—instead, it uses a bidirectional random walk diffusion process to learn node embeddings. This is then combined with a GRU. Its advantage is that it is capable of learning more complex spatial relationships through the diffusion convolution process, and its use of bidirectional flow helps it learn from traffic in both directions. The disadvantage is, again, that it lacks an attention mechanism that would help it more efficiently learn spatial dependencies.

2.3 AGCRN

The Adaptive Graph Convolutional Recurrent Network [1] uses an adaptive graph generation approach, where it modifies the graph as it trains. This is then fed into a GRU. The advantage is that it doesn't need a fixed adjacency matrix, which potentially allows it to generalize to new unseen graphs (i.e. traffic locations). The disadvantage is that this adaptive graph generation process can be very time consuming, which can slow down training and hinder real-time prediction.

2.4 HierAttnLSTM

The Hierarchical Attention LSTM [13] is unique among the ones discussed in this section, because it doesn't involve a GNN—instead, it uses a hierarchy of LSTM cells that each pool their results to the parent LSTM through attention layers. This enables it to effectively attend to information across the time series, with the disadvantage that it is unable to directly learn on the graph structure.

3 Methodology

Referencing the surveyed approaches, I propose to use a combination of a Graph Attention Network (GAT) and a Gated Recurrent Unit (GRU) model (see figure 1).

3.1 Model Architecture

At each timestep, the GAT takes graph data as input and outputs node embeddings. During training, a dropout layer is applied to the output node embeddings to provide additional regularization. The GAT employs an attention mechanism [8, 10]:

$$\mathbf{x}_i' = \alpha_{i,i}\mathbf{W}_s\mathbf{x}_i + \sum_{j\in\mathcal{N}(i)} \alpha_{i,j}\mathbf{W}\mathbf{x}_j, \tag{1}$$

$$\alpha_{i,j} = \frac{e^{\mathrm{LeakyReLU}\left(\mathbf{a}_s^\top \mathbf{W}_s\mathbf{x}_i + \mathbf{a}^\top \mathbf{W}\mathbf{x}_j\right)}}{\sum_{k\in\mathcal{N}(i)\cup\{i\}} e^{\mathrm{LeakyReLU}(\mathbf{a}_s^\top \mathbf{W}_s\mathbf{x}_i + \mathbf{a}^\top \mathbf{W}\mathbf{x}_k)}} \tag{2}$$

These learnable attention weights allow the GAT to attend differently to different neighbors, which enables more complex graph relationships to be captured.

The model also includes a skip connection for the GAT, which can improve feature propagation and prevent overfitting, so as to help the model generalize better [6]. Skip connections have also been shown to improve training speed for GNNs [12].

$$\mathbf{x}_i'' = \mathrm{ReLU}\left(\mathrm{GAT}\left(\mathbf{x}_i\right) + \mathbf{W}_{\mathrm{proj}}\mathbf{x}_i\right) \tag{3}$$

Then, these node embeddings are passed to the GRU. Finally, the output of the GRU is projected back into the input dimensions with a fully connected layer. The resulting output and the hidden state from the GRU are passed back into the model at the next timestep. MSE loss is used because this is a regression problem.

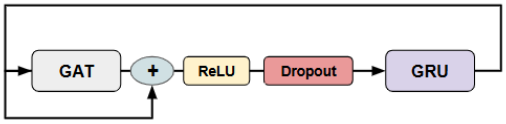


Figure 1: GAT-GRU cell

3.2 Motivation and Comparison to Existing Architectures

The choice of model architecture is motivated by a very important consideration: not all roads are created equal. A GAT can learn to attend to roads that have a greater impact on the traffic at a given location, and thus better capture the spatial dependencies of traffic.

The surveyed GNN architectures above all lack this attention mechanism, with the possible exception of AGCRN [1], which uses an adjacent but similar approach where it modifies the graph as it trains to learn which neighbors are important. However, my proposed methodology is likely to be much less time-intensive to train and deploy than the AGCRN, because modifying the graph on the fly is a time consuming operation. The HierAttnLSTM [13] has an attention mechanism, but it is applied over the temporal domain rather than the spatial domain, so it does not directly learn what neighbors have a greater impact on the traffic at a given node.

My proposed architecture also adds additional regularization through the skip connection on the GAT and the dropout layer. This improves its capability to generalize on unseen data.

A major point of similarity is the use of a GRU, which is also present in most of the other architectures. It is able to effectively capture temporal dependencies in traffic, which complements the GAT's focus on spatial relationships.

4 Experiment

4.1 Dataset

I used two benchmark traffic datasets, PEMS04 and PEMS08 [11]. Both are obtained from the Caltrans Performance Measurement System (PeMS) [3]. Each dataset is a time series of weighted directed graphs in 5 minute intervals. PEMS04 spans Jan-Feb 2018 and has data from 307 sensors on 29 roads, for a total of 16992 timesteps, 307 nodes, and 340 edges. PEMS08 spans Jul-Aug 2016 and has data from 170 sensors on 8 roads, for a total of 17856 timesteps, 170 nodes and 295 edges.

Both datasets have 3 features per node (traffic sensor): flow (the number of vehicles that pass through the sensor in the time interval), occupancy (the proportion of time there was a vehicle at the sensor), speed (average speed of vehicles).

4.2 Training details

Preprocessing was carried out to convert the features and adjacency list into a graph. The data was also Z-standardized along the feature axis using the mean and standard deviation of the training data, to bring all the features into a comparable range. Only the mean and standard deviation of the training data was used, to avoid leaking information from the validation and test datasets.

Each dataset was processed into sequences of length 24, representing a 120-minute time window, which was then split in half such that the model would get the first 12 timesteps (60 minutes) as input and predict the next 12 timesteps. In order to also determine the model's performance on even shorter-term traffic forecasting, the dataset was also similarly processed into input-target sequence pairs of 3 timesteps (15 minutes) each [13].

Minibatch gradient descent [2] was carried out with a batch size of 256. The Adam optimizer was used with an initial learning rate of 1e-3 and training was done for 50 epochs, with the model that achieved the best validation loss being selected at the end. The model was trained on each dataset-and-time-window combination for 3 runs each, using the seeds 0, 42, and 1337. Teacher forcing was used to improve training results.

4.3 Results and Comparisons

All results for my model are averaged over 3 runs with uncertainty equal to their standard deviation. Two metrics are used: Mean Absolute Error (MAE) and Root Mean Square Error (RMSE). MAE measures the general accuracy of the model, while RMSE measures how stable the model is (because it is more sensitive to outlier predictions) [9]. These metrics were computed after denormalizing the predicted values using the previously computed mean and standard deviation of the training data.

The results are compared against those achieved by four existing models: HierAttnLSTM [13], DCRNN [7], AGCRN [1], and TGCN [14]. [1]

Table 1: Results and comparison with other models on PEMS04

	15 min		60 min	
	MAE	RMSE	MAE	RMSE
GAT-GRU	$\mathbf{8.512 \pm 0.182}$	$\mathbf{20.694 \pm 0.238}$	$\mathbf{12.062 \pm 0.372}$	$\mathbf{28.402 \pm 0.087}$
HierAttnLSTM	9.079	22.766	9.168	22.844
AGCRN	18.132	29.221	19.851	31.965
DRCNN	19.581	31.125	24.864	39.228
TGCN	21.678	34.635	29.062	44.794

Table 2: Results and comparison with other models on PEMS08

	15 min		60 min	
	MAE	RMSE	MAE	RMSE
GAT-GRU	$\mathbf{6.750 \pm 0.086}$	$\mathbf{16.368 \pm 0.169}$	$\mathbf{9.792 \pm 0.115}$	$\mathbf{23.639 \pm 0.127}$
HierAttnLSTM	8.375	20.356	9.215	22.320
AGCRN	14.146	22.241	16.427	26.557
DCRNN	15.139	23.476	19.345	30.058
TGCN	17.348	25.934	23.417	34.694

Observe from tables 1 and 2 that the model performs better on the 15-minute time window than the 60-minute time window. This is expected, because a shorter time window means the model doesn't have to predict as far into the future. Since outputs are fed into the model as the input for the next step, errors will propagate, so a shorter time window is advantageous. Also, it can be seen that the model performs better on PEMS08 than PEMS04. This is likely because PEMS08 has a smaller graph, with less nodes and edges, which makes it easier for the model to learn.

As the results demonstrate, the GAT-GRU architecture generally outperforms the other models (i.e. lower MAE and RMSE). In particular, it outperforms HierAttnLSTM on the 15-minute window, and performs only very slightly worse than it on the 60-minute window. For all other models shown here, it significantly outperforms them for all dataset-window combinations on both MAE and RMSE. This demonstrates that the architecture is able to effectively learn to forecast traffic and shows the importance of the attention mechanism when capturing the spatial dependencies of traffic data.

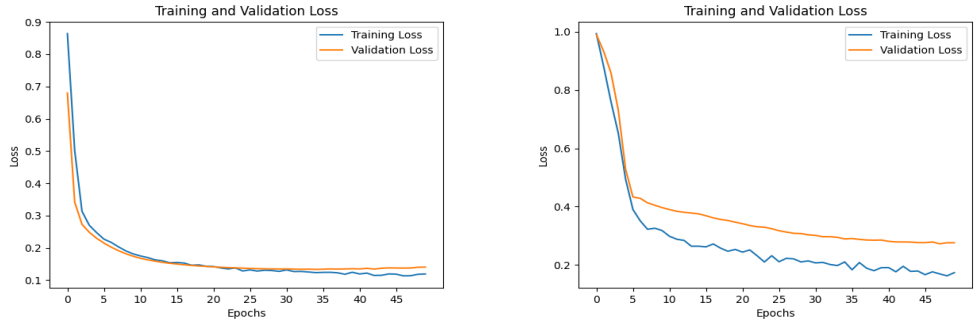
While no explicit ablation studies were carried out, the effectiveness of the GAT can be highlighted by comparing it to the TGCN model: the main difference between my proposed model and TGCN is the use of a GAT (with added regularization) instead of GCN, suggesting that the attention mechanism is hugely important to traffic forecasting.

4.4 Further Analysis

4.4.1 Training/Validation Loss Curves

Observe in figure 2 that the training and validation loss curves are consistently close for 15-minute window traffic prediction, but there is a gap when for 60-minute window prediction. This suggests that the model is able to generalize better for short-term forecasts, which is to be expected because predicting further into the future is an inherently more challenging task.

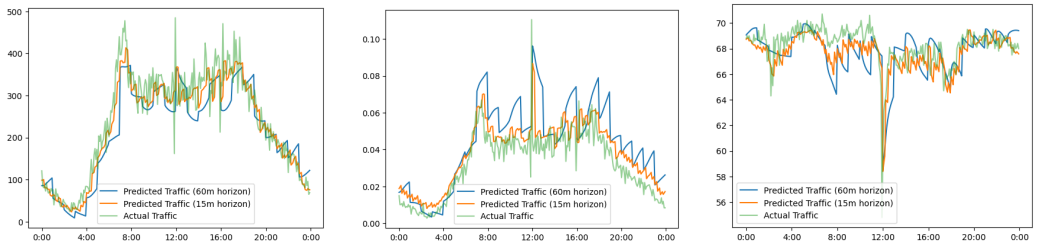
[1] Due to the absence of pretrained models for PEMS04/PEMS08 for most of these architectures, their performance statistics have been taken from the *Network Level Spatial Temporal Traffic State Forecasting with Hierarchical Attention LSTM (HierAttnLSTM)* paper [13].



(a) 15 min window (b) 60 min window

Figure 2: Loss curves on PEMS04 with seed 0

4.4.2 Model Predictions



(a) Flow (b) Occupancy (c) Speed

Figure 3: Predictions for node 0 in a 24-hour period (seed 0)

Two models, both trained on seed 0, but one with a 15-minute horizon and the other with a 60-minute horizon, were used to predict values for an entire day by only looking at the past 15/60 minutes of traffic data. As shown in figure 3, both models don't do as well at peaks/troughs or at abrupt fluctuations, suggesting that while the GAT-GRU architecture is able to learn to predict traffic, it is less capable of predicting unexpected changes in traffic flow/occupancy/speed.

Observe that the 15-minute version consistently outperforms the 60-minute version: its predictions are closer to the actual value, and it is better able to deal with abrupt changes in traffic. The lower "resolution" of the 60-minute version can actually be observed in the figures, where we see a sawtooth pattern roughly corresponding to the 1 hour intervals.

5 Conclusion and Future Works

Experiments have shown that the GAT-GRU model can outperform state-of-the-art models at traffic forecasting, especially for short time horizons (15 minute forecasting). This work demonstrates the impact of attention for traffic prediction, and affirms that hybrid graph convolution and recurrent neural networks are well-suited for spatio-temporal graph regression problems such as traffic forecasting.

The GAT-GRU model has been shown to perform better at short-term forecasting, suggesting that a more powerful model than the GRU may be required to capture temporal relationships in longer sequences, such as an LSTM or a Transformer. This could be an interesting future research direction.

6 Reproducibility

All code and experiments (in Jupyter notebook form) may be found at `https://github.com/justinlam19/gat-gru`

References

[1] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang. Adaptive Graph Convolutional Recurrent Network for Traffic Forecasting, 2020. URL https://arxiv.org/abs/2007.02842.

[2] D. Bertsekas. Incremental least squares methods and the extended Kalman filter. In *Proceedings of 1994 33rd IEEE Conference on Decision and Control*, volume 2, pages 1211–1214 vol.2, 1994. doi: 10.1109/CDC.1994.411166.

[3] C. Chen, K. Petty, A. Skabardonis, P. Varaiya, and Z. Jia. Freeway Performance Measurement System: Mining Loop Detector Data. *Transportation Research Record*, 1748(1):96–102, 2001. doi: 10.3141/1748-12. URL https://doi.org/10.3141/1748-12.

[4] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation, 2014. URL https://arxiv.org/abs/1406.1078.

[5] Elmahy. PEMS dataset. https://www.kaggle.com/datasets/elmahy/pems-dataset, 2021. URL https://www.kaggle.com/datasets/elmahy/pems-dataset. Accessed: 2024-12-10.

[6] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition, 2015. URL https://arxiv.org/abs/1512.03385.

[7] Y. Li, R. Yu, C. Shahabi, and Y. Liu. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting, 2018. URL https://arxiv.org/abs/1707.01926.

[8] P. Team. conv.GATConv. https://pytorch-geometric.readthedocs.io/en/stable/generated/torch_geometric.nn.conv.GATConv.html, 2024. URL https://pytorch-geometric.readthedocs.io/en/stable/generated/torch_geometric.nn.conv.GATConv.html. Accessed: 2024-12-10.

[9] V. Trevisan. Comparing Robustness of MAE, MSE and RMSE, 2022. URL https://towardsdatascience.com/comparing-robustness-of-mae-mse-and-rmse-6d69da870828. Accessed: 2024-12-12.

[10] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph Attention Networks, 2018. URL https://arxiv.org/abs/1710.10903.

[11] J. Wang, J. Jiang, W. Jiang, C. Li, and W. X. Zhao. LibCity: An Open Library for Traffic Prediction. In *Proceedings of the 29th International Conference on Advances in Geographic Information Systems*, SIGSPATIAL '21, page 145–148, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450386647. doi: 10.1145/3474717.3483923. URL https://doi.org/10.1145/3474717.3483923.

[12] K. Xu, M. Zhang, S. Jegelka, and K. Kawaguchi. Optimization of Graph Neural Networks: Implicit Acceleration by Skip Connections and More Depth, 2021. URL https://arxiv.org/abs/2105.04550.

[13] T. T. Zhang. Network level spatial temporal traffic forecasting with hierarchical attention LSTM (HierAttnLSTM). *Digital Transportation and Safety*, 0(0):1–13, 2020. ISSN 2837-7842. doi: 10.48130/dts-0024-0021. URL http://dx.doi.org/10.48130/dts-0024-0021.

[14] L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, and H. Li. T-GCN: A Temporal Graph Convolutional Network for Traffic Prediction. *IEEE Transactions on Intelligent Transportation Systems*, 21(9):3848–3858, Sept. 2020. ISSN 1558-0016. doi: 10.1109/tits.2019.2935152. URL http://dx.doi.org/10.1109/TITS.2019.2935152.